

Этот документ объясняет преподавателю, какие возможности есть в программе и как они реализованы. Никакого кода — только описание алгоритмов и логики простыми словами.

# 1. Генерация пароля

## Что делает

Пользователь вводит секретную фразу (сид), выбирает сервис (например, google или github), настраивает длину и набор символов, нажимает кнопку "Создать" — и получает пароль.

## Как работает

Программа собирает все данные с экрана: берёт текст из поля "Сид", текст из поля "Сервис", число из поля "Длина", и смотрит, какие галочки стоят (заглавные, строчные, цифры, спецсимволы). Из отмеченных галочек составляется алфавит — строка из всех разрешённых символов.

Затем программа проверяет: если сид пустой — показывает розовую подсветку и ошибку "Введите сид". Если ни одна галочка не отмечена — тоже ошибка. Если всё в порядке — запускается алгоритм HMAC-SHA256 (про него дальше).

После того как HMAC выдал пароль, программа смотрит, включена ли опция "Первая заглавная" — если да, первый символ делается большой буквой. Если включена "Последняя цифра" — последний символ заменяется на цифру (тоже детерминированно, через HMAC).

Готовый пароль появляется в поле, автоматически копируется в буфер обмена, а если сервис был введён вручную — он сохраняется в список для будущего использования.

## 2. Алгоритм HMAC-SHA256

### Что это такое

HMAC-SHA256 — это криптографический алгоритм, который принимает секретный ключ и сообщение, а возвращает уникальную подпись (32 байта, то есть 256 бит). Этот алгоритм детерминированный: если ключ и сообщение одинаковые — подпись всегда будет одинаковой.

### Как это используется в программе

Ключом HMAC становится сид пользователя (его секретная фраза). Сообщением — название сервиса, длина пароля и номер шага (счётчик). Из полученной подписи берутся отдельные байты, и каждый байт превращается в символ из алфавита.

Как байт становится символом: берётся остаток от деления значения байта на длину алфавита. Например, если алфавит из 10 символов, а байт равен 42, то  $42 \% 10 = 2$  — берётся третий символ алфавита.

Важная деталь: для равномерности некоторые байты отбрасываются. Если байт слишком большой (из-за чего остаток от деления был бы неравномерным), он пропускается. Это гарантирует, что все символы алфавита выпадают с одинаковой вероятностью.

Если пароль длиннее 32 символов, одной подписи не хватает. Тогда увеличивается счётчик, и вычисляется новая подпись с тем же ключом, но другим сообщением. Процесс повторяется, пока не наберётся нужное количество символов.

## 3. Генерация логина

### Что делает

Создаёт короткий (6 символов) логин для сервиса. Логин специально сделан таким, чтобы его можно было запомнить, а не просто копировать.

### Как работает

Берётся хэш SHA-256 от трёх склеенных частей: сид + слово "login" + название сервиса. Из хэша берутся первые 6 байт, каждый байт превращается в символ из специального алфавита.

Алфавит для логина состоит из 31 символа: строчные буквы (без l и o, чтобы не путать с единицей и нулём) и цифры (без 0 и 1). Так логин легко прочитать и продиктовать.

Логин, как и пароль, детерминированный: одни и те же сид и сервис всегда дают один и тот же логин.

## 4. Оценка надёжности пароля

### Что делает

Под полем с паролем показываются три полосы и текст, которые визуальнo оценивают, насколько пароль надёжен.

### Как работает

Функция оценки получает два числа: длину пароля и количество включённых наборов символов (от 1 до 4). По ним вычисляется уровень:

- Слабый (розовая полоска): длина меньше 10 или наборов меньше 3
- Средний (малиновая полоска): длина от 10 и наборов от 3
- Сильный (сиреневая полоска): длина от 14 и наборов 4

В зависимости от уровня закрашивается разное количество полосок (1, 2 или 3 из трёх) и показывается текст с рекомендацией. Переход между уровнями анимирован.

## 5. Инверсия (Первая заглавная и Последняя цифра)

### Что делает

Две дополнительные настройки в блоке "Расширенные", которые изменяют уже готовый пароль.

### Как работает "Первая заглавная"

После того как HMAC сгенерировал пароль, программа берёт первый символ и делает его заглавной буквой. Остальные символы не меняются.

### Как работает "Последняя цифра"

Программа вычисляет дополнительный HMAC от сида с ключом "\_lastdig" и названием сервиса. Из подписи берётся первый байт, от него остаток от деления на 10 — получается цифра от 0 до 9. Последний символ пароля заменяется на эту цифру.

Важно: цифра вычисляется детерминированно, то есть для одних и тех же данных всегда будет одна и та же цифра. Обе опции накладываются поверх готового пароля, не влияя на основную HMAC-генерацию.

## 6. Список сервисов

### Что делает

Рядом с полем ввода сервиса есть стрелка "список". При нажатии открывается панель с популярными сервисами и теми, которые пользователь добавлял раньше.

### Как работает

В программе есть предустановленный список популярных сервисов (google, github, vk, yandex и другие). У каждого сервиса задан цвет и буква для логотипа.

Когда пользователь вводит свой сервис и создаёт пароль, название автоматически добавляется в отдельный список. Этот список хранится в памяти браузера и восстанавливается при следующем запуске.

Панель строится так: сначала показываются популярные сервисы (с цветными кружками-логотипами), потом пользовательские (с серыми кружками и знаком вопроса). По клику на любой сервис его название подставляется в поле ввода. По правому клику на пользовательском сервисе он удаляется из списка.

## 7. Пресеты (преднастройки)

### Что делает

Выпадающий список с тремя готовыми вариантами настроек: Минимальный, Стандартный, Максимальный. Быстро выставляют длину и набор символов.

### Как работает

Когда пользователь выбирает вариант, программа проверяет, какой выбран, и устанавливает соответствующие значения:

- Минимальный: длина 8, буквы + цифры, без спецсимволов
- Стандартный: длина 10, всё кроме спецсимволов
- Максимальный: длина 16, все наборы

После применения интерфейс сразу обновляется — меняются галочки и значение длины.



## 8. Экспорт и импорт настроек

### Что делает

Кнопки "Экспорт" и "Импорт" в верхней части программы. Позволяют сохранить настройки в файл и загрузить их обратно.

### Как работает экспорт

Программа собирает в один объект все настройки: какие галочки стоят, какие опции инверсии включены, список сервисов, длину. Преобразует это в JSON-формат (текстовый формат для обмена данными) и скачивает файл `passgen_export.json`.

### Как работает импорт

Пользователь выбирает JSON-файл на своём компьютере. Программа читает его, проверяет, что внутри корректные данные, и восстанавливает настройки: галочки, сервисы, инверсию, длину.

### Важно

Сид никогда не сохраняется в файл экспорта — это сделано намеренно. Если бы сид сохранялся, при передаче файла кому-то ещё он бы получил доступ ко всем паролям. Сид должен оставаться только в голове пользователя.

## 9. Сохранение настроек между запусками

### Что делает

Когда пользователь закрывает и снова открывает программу, все его настройки (галочки, длина, список сервисов) восстанавливаются.

### Как работает

При каждом изменении (нажатии на галочку, добавлении сервиса, изменении длины) программа сохраняет текущие настройки в `localStorage` — это специальное хранилище в браузере, которое не очищается при закрытии страницы.

При запуске программа проверяет, есть ли сохранённые настройки в этом хранилище, и если есть — восстанавливает их и обновляет интерфейс.

Сами пароли никогда не сохраняются — в этом и смысл программы: пароль каждый раз вычисляется заново.

## 10. Анимации интерфейса

### Что делает

Все элементы программы появляются плавно, переключатели анимированно меняют состояние, панели раскрываются без рывков.

### Как работает

Используются CSS-анимации (стили браузера):

- Карточки при загрузке плавно подплывают вверх с прозрачности
- Рамка при ошибке пульсирует розовым цветом
- Панель сервисов раскрывается плавно (max-height анимируется)
- Кружки-переключатели при нажатии делают "пружинку" (увеличиваются и возвращаются)
- Кнопки при нажатии слегка уменьшаются

Анимации сделаны через CSS-свойство transition и @keyframes, никаких сторонних библиотек не используется.

# 11. Серверная часть (app.py)

## Что делает

Файл app.py запускает простой веб-сервер на Python, который показывает HTML-файл программы.

## Как работает

Сервер запускается на порту 8765 и раздаёт файлы из своей папки. Если порт занят, автоматически берётся следующий свободный порт.

После запуска сервер пытается открыть программу в браузере в режиме приложения (без адресной строки, как отдельное окно). Для этого перебираются установленные браузеры: Google Chrome, Chromium, Brave, Edge (в режиме --app) или Firefox (в отдельном окне). Если ни один не найден — открывается в браузере по умолчанию.

Сервер работает в фоновом потоке и ждёт, пока пользователь нажмёт Ctrl+C в терминале, чтобы корректно завершиться.

## Итог

Программа полностью работает в браузере, не требует установки библиотек, работает на Windows и Linux. Пароли не хранятся, а вычисляются через HMAC-SHA256 на стороне пользователя, что обеспечивает безопасность и детерминированность.